Deploying the Medplum Healthcare Platform on Microsoft Azure

An Enterprise-Grade Cloud Infrastructure Built with Terraform and Kubernetes

A Final Project for EPITA | By Muhammad Naveed, Naveed Riaz, Muhabbat Ali, Muntazir Abbas

NotebookLM

A Production-Ready Healthcare Platform, Deployed Successfully on Azure

8 Containerised Services

Deployed on a scalable **Azure Kubernetes Service (AKS)** cluster.

~25 Azure Resources

Automatically provisioned and managed via Terraform for full Infrastructure as Code (IaC).

Secure by Design

Implemented with **network isolation** in a private VNet, Network Security Groups, and managed identities.

Publicly Accessible

Full DNS management for a custom domain (**medplum.fun**) routing to all services.

Stateful & Persistent

Configured **Azure Managed Disks** for stateful services like **PostgreSQL**, ensuring data persistence.

Integrated Observability

Established foundational **monitoring** and **logging** with **Azure Monitor** and **Log Analytics**.

NotebookLM

Built to Meet Demands for Scalability, Security, and Maintainability

Project Goals

- ✓ **Scalability**: Handle varying loads with AKS auto-scaling.
- ✓ **High Availability**: Ensure service uptime with multiple replicas and failover.
- ✓ **Security**: Enforce network isolation, access controls, and encryption.
- ✓ **Cost Efficiency**: Optimise resource usage to minimise operational costs.
- ✓ **Maintainability**: Utilise IaC for reproducible, version-controlled deployments.
- ✓ **Monitoring**: Achieve comprehensive observability into the entire stack.

Core Technology Stack

Cloud Provider: Microsoft Azure

IaC & Automation: Terraform, Helm, Bash Scripting

Orchestration: Azure Kubernetes Service (AKS)

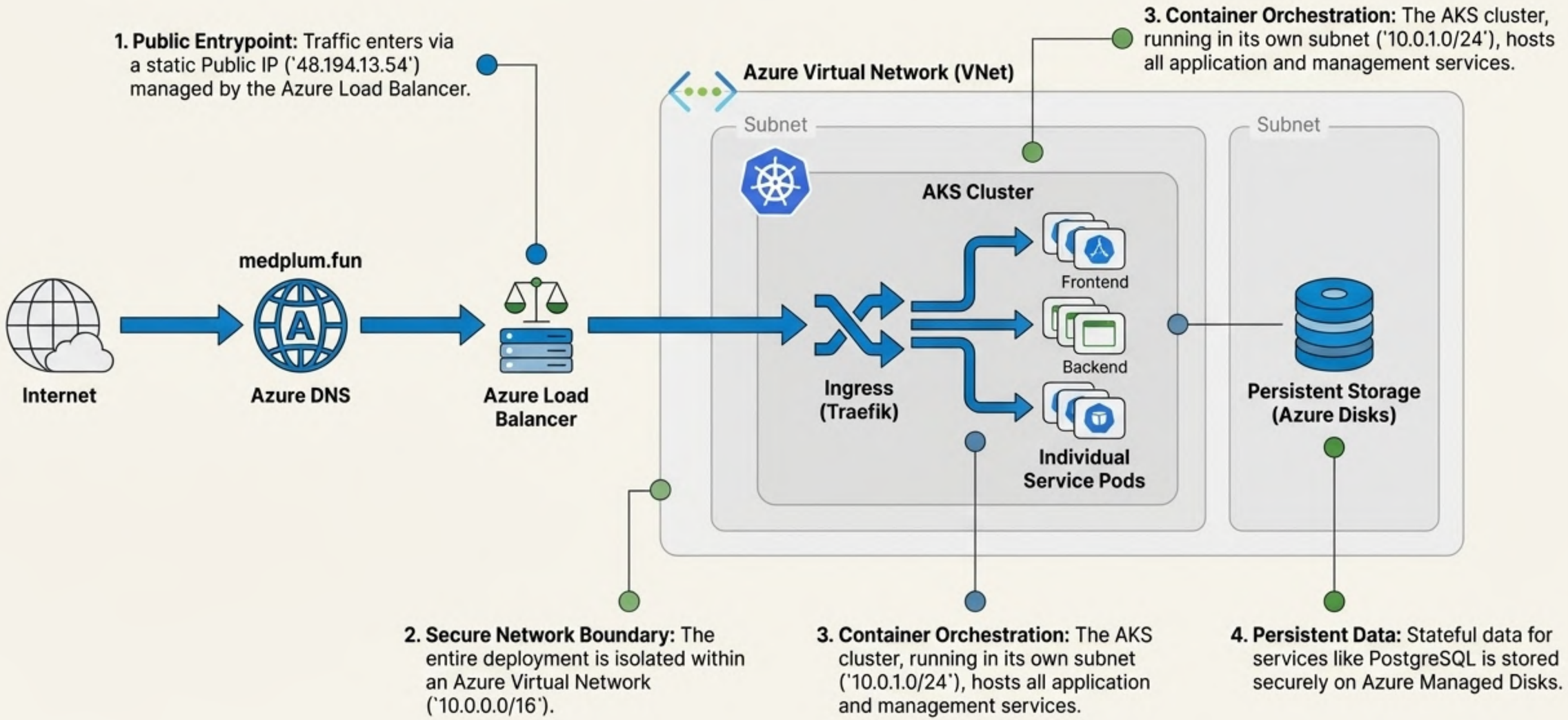
Key Azure Services:

- Azure Container Registry (ACR)
- Virtual Network (VNet) & NSG
- Azure DNS
- Azure Load Balancer
- Azure Managed Disks
- Azure Monitor & Log Analytics

Application Services: PostgreSQL, Portainer, Prometheus, Kafka, Vault

NotebookLM

The Architecture: A Tour of the Medplum Platform on Azure



NotebookLM

The Foundation: A Segmented and Secured Virtual Network

VNet & Subnet Strategy

`medplum-stack-dev-vnet`
10.0.0.0/16

`medplum-stack-dev-aks-subnet`
10.0.1.0/24

Houses the AKS nodes and pods, enabling advanced Azure CNI networking.

`medplum-stack-dev-private-subnet`
10.0.2.0/24

Reserved for internal services and storage endpoints, isolating them from the main cluster network.

Network Security Group (NSG) Rules

`medplum-stack-dev-aks-nsg`

Principle: Default-deny, with explicit allow rules for necessary traffic.

HTTP Allow — Priority 100: Allow Port 80

HTTPS Allow — Priority 110: Allow Port 443

API — Priority 200: Allow Port 443

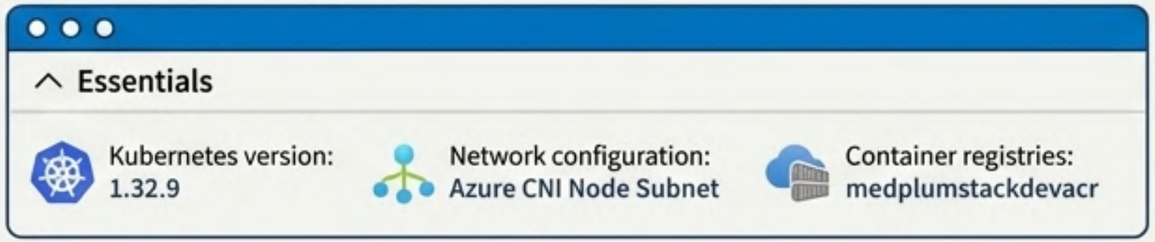
NotebookLM

The Engine: Orchestrating Services with Azure Kubernetes Service (AKS)

AKS Cluster Configuration

`medplum-stack-dev-aks`

- Kubernetes Version: `1.32.9`
- Node Pool:
  - VM Size: `Standard_B2s` (2 vCPUs, 4GB RAM)
  - Auto-scaling: Enabled (Min: 1, Max: 2 nodes)
- Key Features Enabled:
  - **Azure CNI Networking**: For seamless VNet integration and network policy enforcement.
  - **Managed Identity**: For secure, credential-less access to other Azure resources like ACR.
  - **Azure Monitor Integration**: Providing deep insights into container performance.



The Rationale: Why AKS?

- **Managed Control Plane**: Eliminates the overhead of managing Kubernetes master nodes, allowing focus on applications.
- **Native Azure Integration**: Provides seamless integration with Azure AD, ACR, Azure Monitor, and VNet.
- **Built-in Security:** Leverages Azure's robust security and compliance features, including managed identities and RBAC.

NotebookLM

The Supply Chain: Managing Container Images and Persistent Data

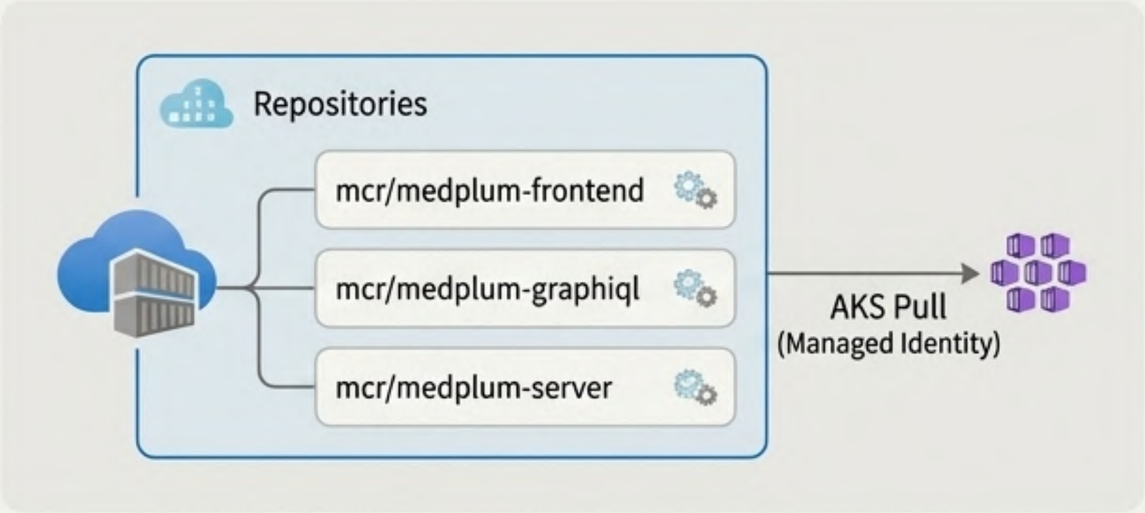
Azure Container Registry (ACR)

Purpose: A private, secure registry for all application container images.

Registry Name: medplumstackdevacr (SKU: Basic)

Key Repositories: medplum-frontend, medplum-backend, medplum-graphql.

Integration: AKS authenticates and pulls images using its system-assigned managed identity with the **AcrPull** role, eliminating the need for stored credentials.



Azure Managed Disks for Persistence

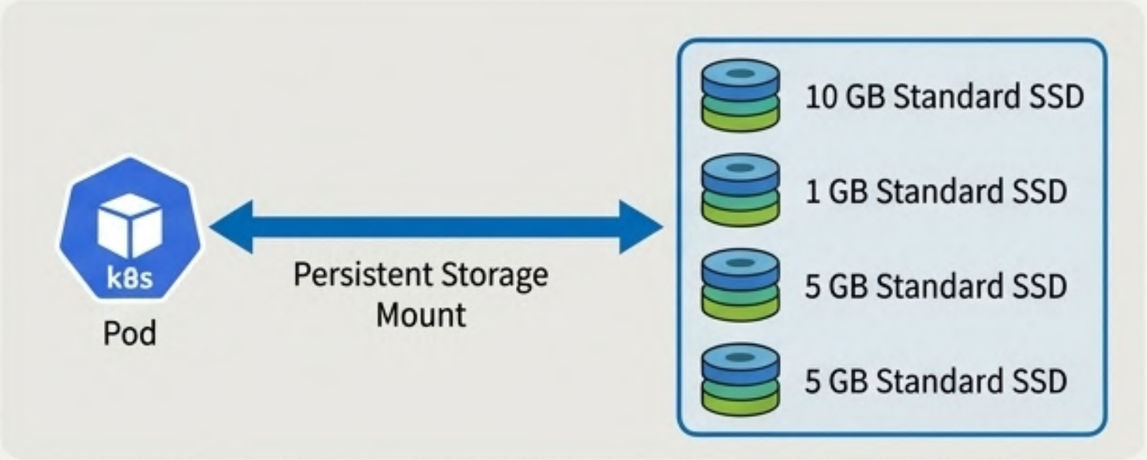
Problem: Container filesystems are **ephemeral**. Stateful services like PostgreSQL require **persistent storage**.

Solution: Kubernetes **Persistent Volume Claims (PVCs)** dynamically provision **Azure Managed Disks**.

Example PVCs:

- **PostgreSQL: 10 GB** Standard SSD
- **Prometheus:** 1 GB Standard SSD

Storage Class: azure-disk-standard with **Reclaim Policy: Retain** to ensure data is preserved even if a pod is deleted.



NotebookLM

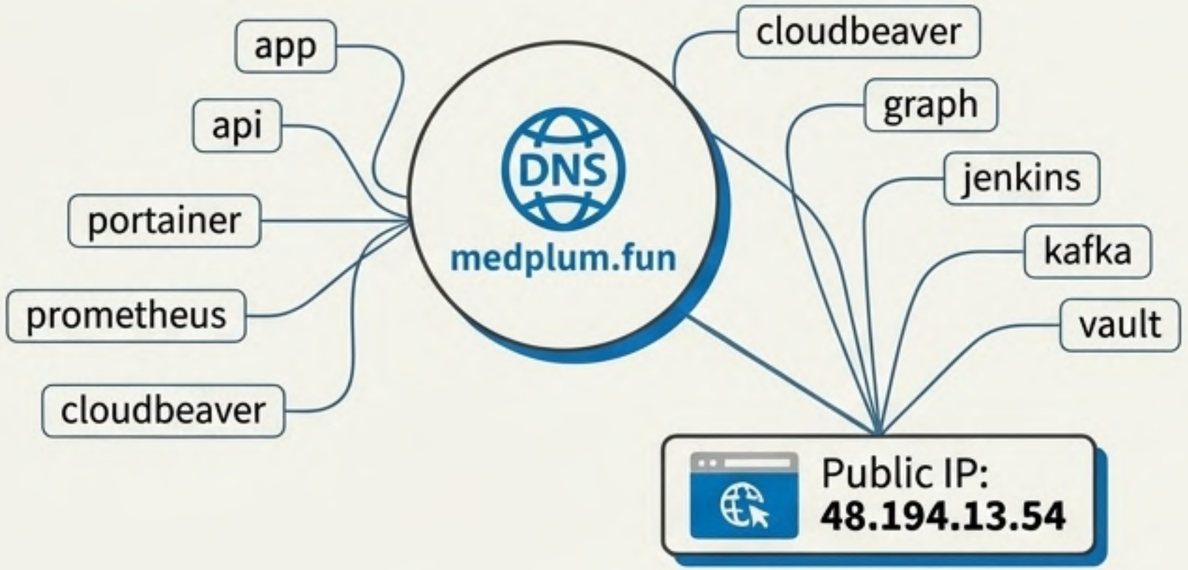
The Public Interface: Connecting Users via DNS and Load Balancing

Azure DNS Zone Management

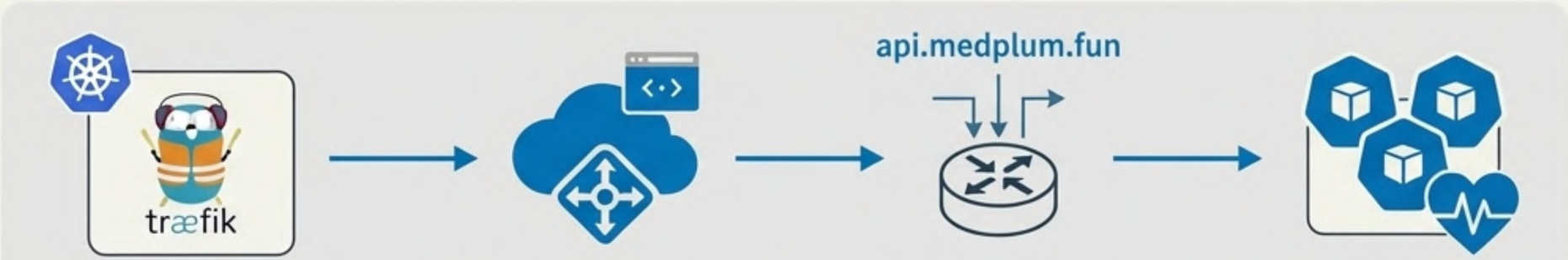
Domain: medplum.fun

Process: Nameservers at the Hostinger registrar were delegated to Azure DNS. All DNS records are now managed within Azure.

Configuration: Multiple `A` records point service subdomains to the single public IP of the Azure Load Balancer (48.194.13.54).



Azure Load Balancer & Traefik Ingress



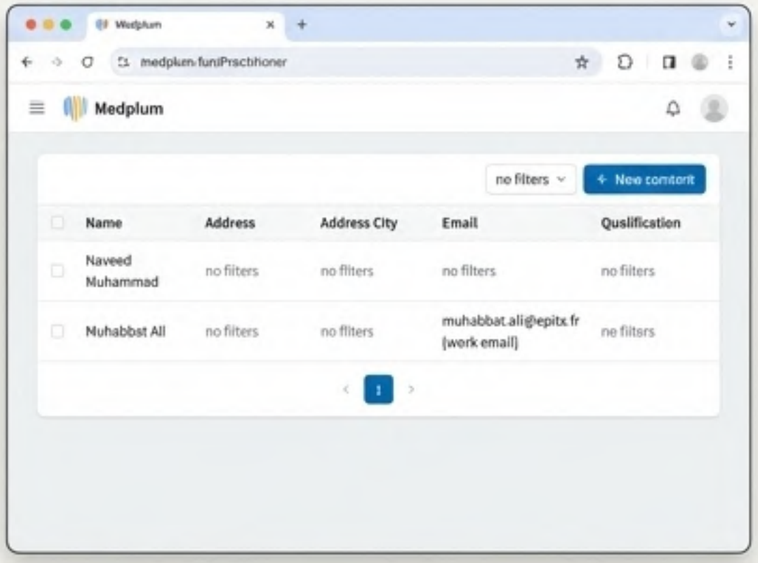
1. Traefik Ingress Controller exposed as `**LoadBalancer**` type.
2. AKS automatically provisions a Standard SKU Load Balancer with a static public IP.
3. Traefik inspects incoming requests (e.g., `api.medplum.fun`) and routes them to the correct internal service pod.
4. The Load Balancer ensures traffic is only sent to healthy nodes.

NotebookLM

The Proof: A Live, Functioning, and Manageable Platform

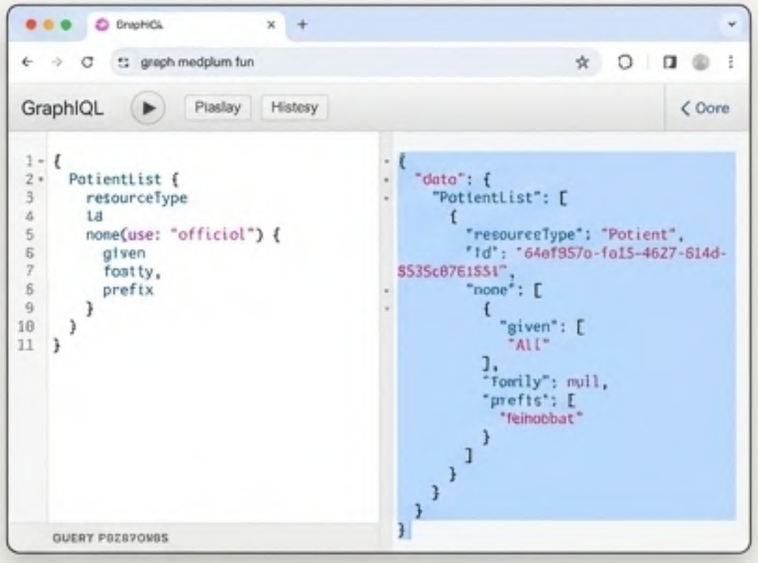
Medplum Frontend Application

Source Sans Pro Semibold #333333



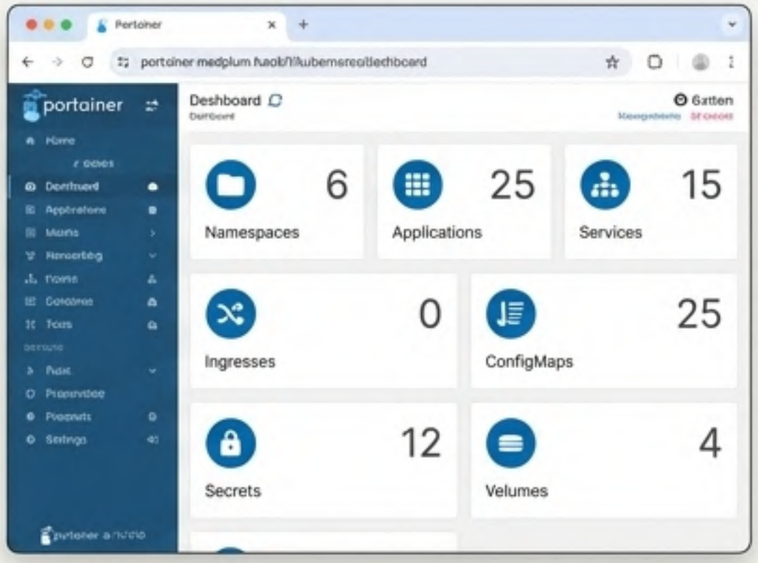
GraphQL API Endpoint

Source Sans Pro Semibold #333333



Portainer Management Dashboard

Source Sans Pro Semibold #333333



Medplum Frontend Application

Source Sans Pro Semibold #333333

GraphQL API Endpoint

Source Sans Pro Semibold #333333

Portainer Management Dashboard

Source Sans Pro Semibold #333333

Live screenshots confirming successful deployment and accessibility of the frontend, backend API, and container management interfaces.

Source Sans Pro Regular #666666

NotebookLM

The Blueprint: Repeatable Infrastructure via Terraform Automation

1. Code Definition
(HCL files in git)

2. terraform init
(Prepare environment)

3. terraform plan
(Review changes, ~25 resources)

4. terraform apply
(Provision Azure resources)

5. Outputs
(Cluster credentials, IPs, etc.)

- **Single Source of Truth:** The entire Azure infrastructure, from the VNet to the AKS cluster, is defined declaratively in Terraform configuration files.
- **Automated Provisioning:** The **terraform apply** command orchestrates the creation of approximately 25 interconnected Azure resources in the correct order.
- **Idempotent & Consistent:** Rerunning the process ensures the infrastructure converges to the desired state, eliminating configuration drift and enabling disaster recovery.
- **Key Terraform Outputs:** The process automatically provides critical connection information, including the `az aks get-credentials` command, Load Balancer IP, and ACR login server.

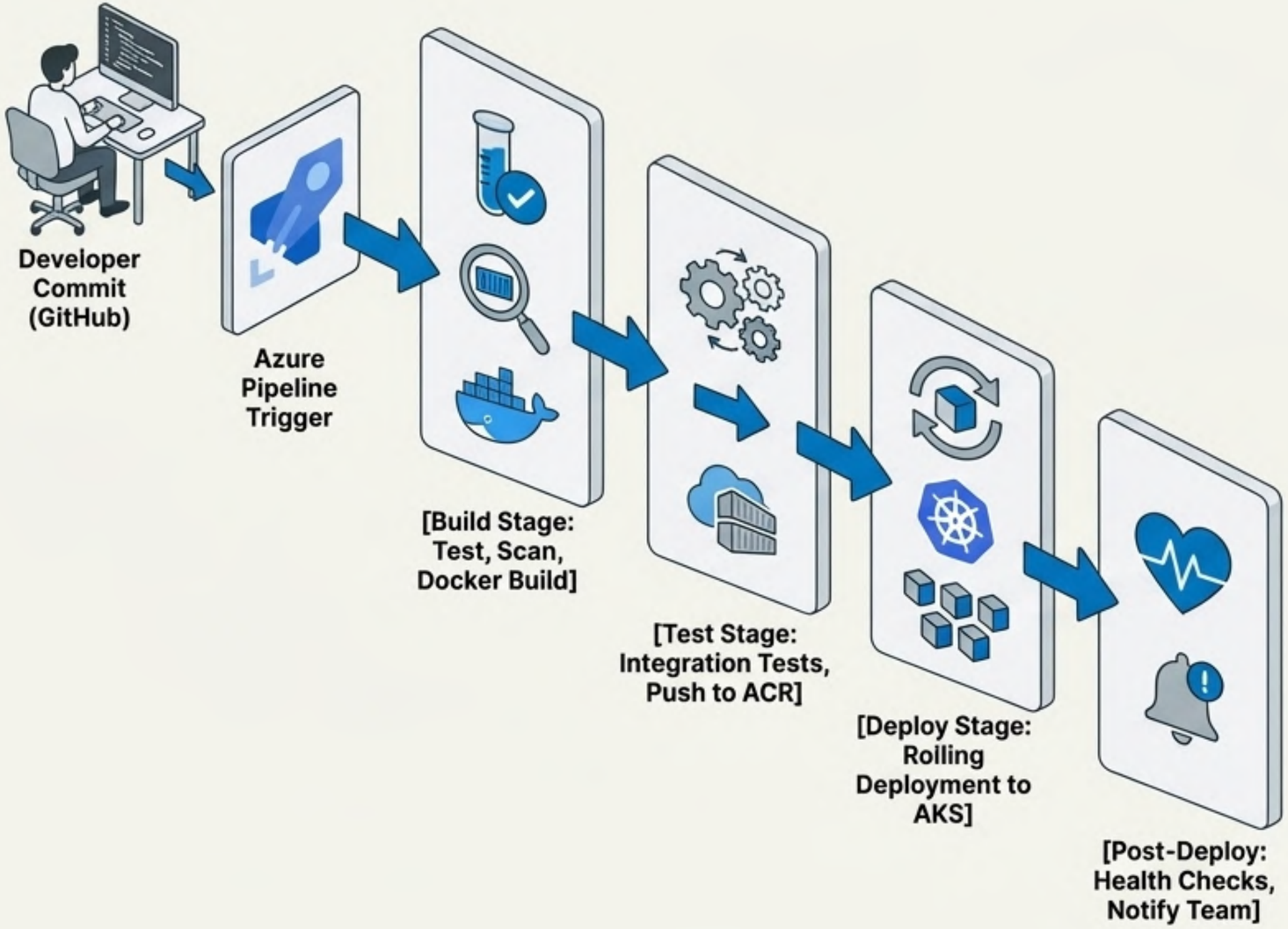
NotebookLM

Lessons from the Field: Turning Challenges into Engineering Insights

The Challenge	The Solution	The Insight
Undersized AKS Nodes	Upgraded nodes from a micro instance to **Standard_B2s** (4GB RAM) to stabilise the cluster.	Always test with realistic workloads before finalising VM sizes.
PostgreSQL Data Loss on Pod Restarts	Implemented a **PersistentVolumeClaim** with an Azure Disk storage class (**reclaimPolicy: Retain**).	Stateful applications in Kubernetes absolutely require persistent external storage.
ACR Image Pull Authentication Failures	Granted the AKS managed identity the **AcrPull** role assignment for the Azure Container Registry.	Managed identities are the superior, more secure method for Azure service-to-service auth.
Inaccessible Services due to NSG Rules	Systematically planned, documented, and tested firewall rules for each required port (e.g., 80, 443).	Network security requires a methodical approach; test connectivity layer by layer.

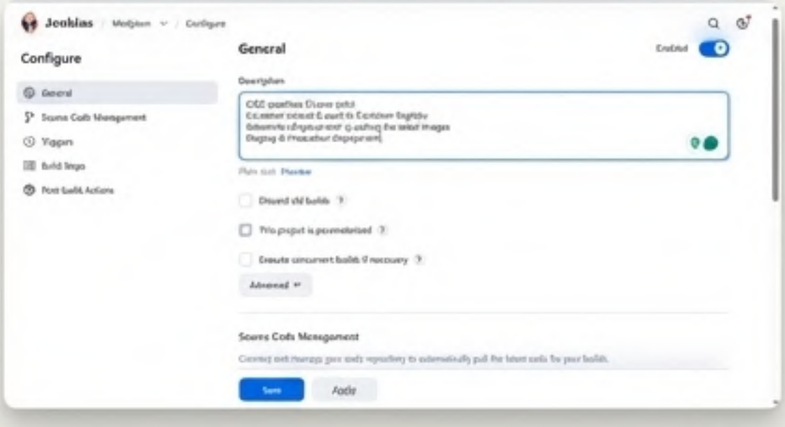
NotebookLM

The Vision Part I: A Fully Automated CI/CD Pipeline



Key Enhancements Planned

- **Automated Testing**: Unit, integration, and end-to-end tests integrated into the pipeline with an 80% code coverage target.
- **Integrated Security Scanning**: Container vulnerability scanning (e.g., Trivy) and secret detection on every build.
- **Advanced Deployment Strategies**: Implementing Blue-Green or Canary releases for zero-downtime deployments and automatic rollbacks on failure.
- **Environment Management**: Separate, gated pipelines for dev, staging, and production environments.



NotebookLM

The Vision Part II: A Robust Data Backup and Recovery Strategy

Azure Blob Storage

Strategy: Automated daily backups of application data and configurations.

Technology: Azure Functions with timer triggers to execute backup logic.

Features:

- Lifecycle policies to move data from **Hot → Cool → Archive tiers**.
- **Geo-redundant storage (GRS)** for cross-region replication.
- **Soft-delete and versioning** to protect against accidental deletion.

Azure Managed Disks (PostgreSQL)

Strategy: Frequent, automated snapshots of the database disk for point-in-time recovery.

Technology: Azure Backup Vault with custom backup policies.

Features:

- **RPO**: 4 hours (via snapshots every 4 hours).
- **RTO**: 2 hours.
- **Incremental snapshots** for cost efficiency.
- **Cross-region restore** capabilities for disaster recovery.

Scheduled Testing

Principle: A backup strategy is only valid if it is **regularly tested**.

Plan: Monthly **automated restore tests** to a staging environment to verify backup integrity and measure restore times.

NotebookLM

Project Outcomes: Technical Mastery and Business Value

Key Learning Outcomes

- **Technical Proficiency:** Deep expertise gained in Azure, Terraform (IaC), Kubernetes, and network architecture.
- **Best Practices Mastered:**
  - The imperative of using IaC for reproducibility.
  - The security benefit of managed identities over credentials.
  - The necessity of planning for data persistence from day one.
  - The importance of comprehensive monitoring and documentation.

Business Value for Healthcare Operations

Scalability: The platform can grow seamlessly with increasing user demand.

Reliability: High-availability design minimises downtime, crucial for healthcare services.

Security & Compliance: A strong foundation of network isolation and encryption protects sensitive patient data and prepares for compliance audits (e.g., HIPAA, GDPR).

Cost-Effectiveness: Optimised resource choices and auto-scaling keep operational costs low.

Maintainability: The IaC approach allows for rapid, safe, and low-cost updates and changes.

NotebookLM

More Than a Deployment: A Blueprint for Continuous Improvement

> ***Infrastructure as Code is not just about automation; it's about creating knowledge that can be shared, version-controlled, and continuously improved. This project embodies that principle, providing both a working system and a blueprint for its future.***

This project successfully demonstrates the creation of an **enterprise-grade platform** on Azure, combining robust architecture with modern **automation practices.** The resulting infrastructure is not a final state but a living, documented, and repeatable **asset** ready for future development and enhancement.

NotebookLM